МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально-науковий інститут інформаційних технологій і робототехніки
Кафедра комп’ютерних та інформаційних технологій і систем

Лабораторна робота № 2

з навчальної дисципліни
"Технології розробки корпоративних Web-додатків"

Виконав:
Студент 402 – ТН
Кузьмін Михайло Олександрович
Перевірив:
Тютюнник Петро Богданович

Полтава 2026

Проєктування архітектури програмного забезпечення та моделювання даних

1. Обґрунтування вибору технологічного стеку

Back-end

Node.js + Express.js

Для реалізації серверної частини застосунку обрано Node.js з фреймворком Express.js. Дане рішення дозволяє використовувати JavaScript як на клієнтській, так і на серверній стороні, що спрощує розробку та підтримку проєкту. Express забезпечує швидке створення REST API, маршрутизацію запитів та інтеграцію з middleware (авторизація, обробка файлів, валідація).

Front-end

React SPA

React обрано для реалізації клієнтської частини інтернет-магазину завдяки компонентному підходу, який спрощує повторне використання елементів інтерфейсу (картка товару, кошик, форма авторизації). SPA-модель забезпечує швидке оновлення інтерфейсу без перезавантаження сторінки, що покращує користувацький досвід.

Система керування базами даних

MongoDB (NoSQL)

MongoDB обрана через гнучку структуру документів, що зручно для каталогу товарів із різними характеристиками (розмір, колір, категорія, ціна). Документна модель добре поєднується з JSON-структурами, які використовуються в Node.js, та дозволяє легко масштабувати систему.

DevOps / Hosting (опціонально)

GitHub — контроль версій і зберігання коду.

Docker — контейнеризація застосунку (опціонально).

Хостинг (Render/Vercel/Railway) — можливість розгортання вебдодатку.

Загальний висновок

Обраний технологічний стек забезпечує швидку розробку сучасного вебзастосунку, хорошу масштабованість, просту інтеграцію між компонентами системи та відповідає вимогам інтернет-магазину одягу.

2. Архітектурне моделювання за методологією C4

C4 LEVEL 1 — System Context Diagram

Рисунок 1 - System Context Diagram

Центральний блок

Online Clothing Store System

- вебдодаток інтернет-магазину
- каталог товарів
- кошик
- замовлення

Актори

Покупець → перегляд і покупка

Адміністратор → керування товарами

Менеджер складу → зміна залишків

Зовнішні системи

- Payment Gateway API
- Delivery Service API

Діаграма контексту показує систему інтернет-магазину як єдину сутність та її взаємодію з користувачами і зовнішніми сервісами.

C4 LEVEL 2 — Container Diagram

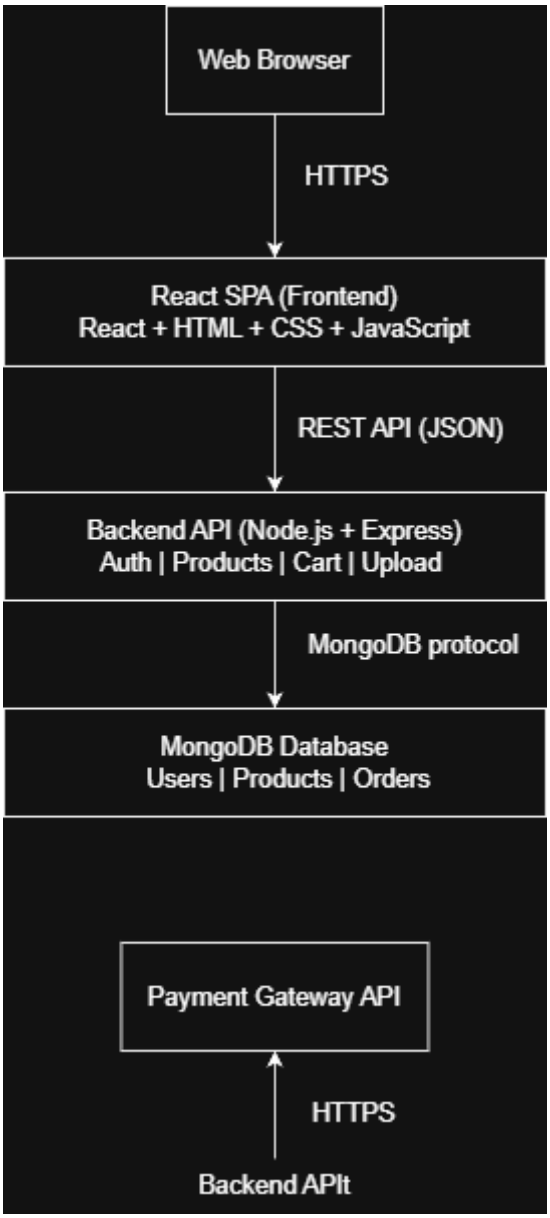

Рисунок 2 - Container Diagram

React SPA

- UI
- Catalog pages
- Cart interface
- Admin panel

Backend API

- JWT Authentication
- Product management
- Cart logic
- Image upload

MongoDB

- Document storage
- NoSQL

Container diagram демонструє фізичне розділення системи на клієнтський застосунок, серверне API та базу даних.

C4 LEVEL 3 — Component Diagram

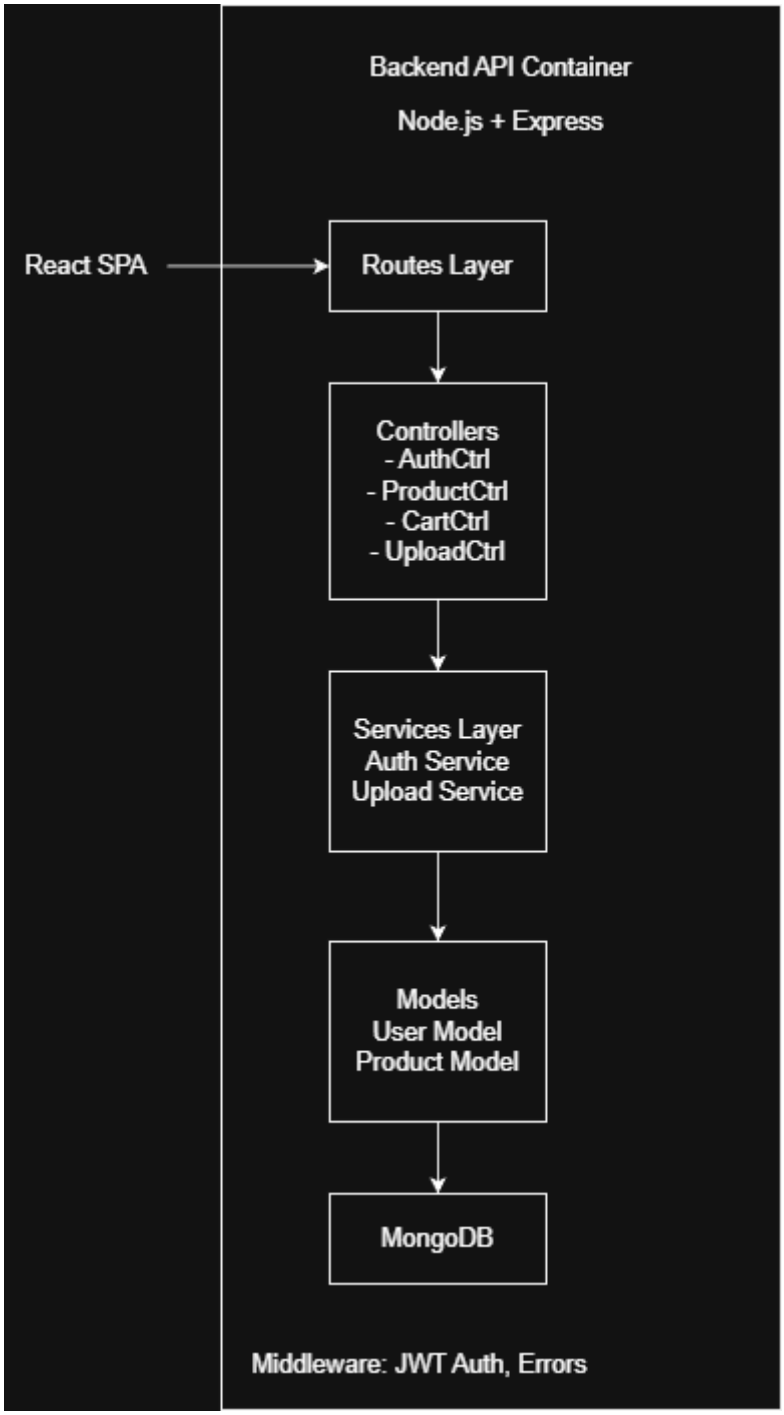

Рисунок 3 - Component Diagram

Компоненти всередині API

Controllers

- AuthController
- ProductController
- CartController

- UploadController

Middleware

- JWT Auth Middleware
- Error Handler

Models

- User Model
- Product Model

Services

- File Upload Service
- Auth Service

Потік

React → Routes → Controllers → Models → MongoDB

Компонентна діаграма деталізує внутрішню структуру API: маршрути, контролери, сервіси, моделі та middleware.

3. Проєктування бази даних (NoSQL — MongoDB)

Для зберігання даних у вебдодатку використовується документоорієнтована база даних MongoDB. Оскільки MongoDB належить до NoSQL систем, структура даних представлена у вигляді колекцій документів. Для логічного моделювання була побудована схема даних (аналог ER-діаграми), яка відображає основні сутності системи та зв'язки між ними.

Рисунок 4 – Проєктування бази даних онлайн-інструментом dbdiagram.io

3.1 Колекція Users

Колекція призначена для зберігання інформації про зареєстрованих користувачів системи.

Основні поля:

- _id — унікальний ідентифікатор користувача;
- name — ім'я користувача;
- email — електронна адреса (унікальна);

- password — пароль (у реальному застосунку зберігається у хешованому вигляді);
- role — роль користувача (покупець або адміністратор);
- createdAt — дата реєстрації.

3.2 Колекція Products

Колекція містить інформацію про товари магазину.

Основні поля:

- _id — ідентифікатор товару;
- name — назва товару;
- image — посилання на зображення;
- category — категорія товару;
- new_price — поточна ціна;
- old_price — попередня ціна;
- available — наявність товару;
- createdAt — дата додавання.

3.3 Колекція Orders

Колекція використовується для зберігання замовлень користувачів.

Основні поля:

- _id — ідентифікатор замовлення;
- userId — посилання на користувача, який зробив замовлення;
- totalPrice — загальна вартість замовлення;
- status — статус замовлення (pending, paid, shipped тощо);
- createdAt — дата створення замовлення.

Один користувач може мати багато замовлень (зв’язок 1:N).

3.4 Колекція OrderItems

Колекція містить товари, що входять до складу конкретного замовлення.

Основні поля:

- _id — ідентифікатор запису;
- orderId — посилання на замовлення;
- productId — посилання на товар;
- quantity — кількість товару;
- price — ціна товару на момент покупки.

Це дозволяє реалізувати зв’язок N:N між замовленнями та товарами.

3.5 Колекція CartItems

Колекція зберігає товари, додані користувачем до кошика.

Основні поля:

_id — ідентифікатор запису;

userId — посилання на користувача;

productId — посилання на товар;

quantity — кількість одиниць товару.

3.6 Стратегія зв’язків

У схемі використовується підхід Referencing, тобто зв’язки між документами реалізуються через збереження ідентифікаторів (ObjectId). Це дозволяє:

- уникнути дублювання даних;
- забезпечити гнучкість структури;
- спростити оновлення інформації про товари;
- підвищити масштабованість системи.

3.7 Загальний висновок

Спроєктована модель даних відповідає вимогам інтернет-магазину одягу та забезпечує зручне зберігання користувачів, товарів, кошиків і замовлень. Використання MongoDB дозволяє легко змінювати структуру документів та масштабувати систему за потреби.